

VLSI Project #2 – State Machine Design – Sam_Sheng

1. Descriptions

1.1 General Description

We are doing team 2's project from the Neuromorphic project that was given to us. This project is a Digital SNN tile, which is responsible for implementing 32 LIF neurons. From the previous project, when an AER event is received the requested neuron is updated from a shared fixed-point arithmetic datapath which updates the state. If the neuron's membrane voltage reaches a threshold value, the tile will produce an outgoing AER spike. The state machine implements control for how the AER event is handled. It sequences capture, arithmetic, state write-back, and output event transmission.

1.2 State Descriptions

IDLE: When the state is IDLE, it waits for AER_IN_REQ to be asserted. When it is asserted, it captures and stores the address (AER_IN_ADDR), neuron_id, and asserts the ACK. After these happen, the state changes to WAIT_INPUT_REQ_LOW.

WAIT_INPUT_REQ_LOW: When the state is WAIT_INPUT_REQ_LOW, it waits for the sender to lower AER_IN_REQ. When the REQ is low, the ACK (AER_IN_ACK) is lowered and the state is changed to READ_NEURON.

READ_NEURON: When the state is READ_NEURON, it saves the next voltage and saves the spike. After these are saved, the state changes to WRITE_NEURON.

WRITE_NEURON: When the state is WRITE_NEURON, it first stores the next voltage in the voltage array for the specific neuron_id given by AER_IN_ADDR. It then checks if a spike was detected, if so it changes the state to SEND_SPIKE, if not it sends the state back to IDLE.

SEND_SPIKE: When the state is SEND_SPIKE, it first checks if the output REQ is low, if so, the output ADDR is set (based on the input address) and the output REQ is asserted. Once the output ACK (AER_OUT_ACK) is high, the output REQ is lowered and the state is changed to WAIT_OUTPUT_ACK_LOW.

WAIT_OUTPUT_ACK_LOW: When the state is WAIT_OUTPUT_ACK_LOW, it checks if the Output ACK is low, and changes the state back to IDLE.

1.3 List of Modules

For the implementation of the state machine for our project we only have one module for our implementation in SystemVerilog. Below is the module block for block2_digital_snn. This module implements the state machine for our project that will implement the Digital SNN tile.

```
module block2_digital_snn #(
    // Supply these values from the testbench for now.

    // Unknown defaults expose a missing test configuration in simulation.

    parameter logic [7:0]    BETA_VALUE    = 8'hxx,

    parameter logic signed [7:0] WEIGHT_VALUE = 8'shx,
```

```

parameter logic signed [11:0] THRESHOLD_VALUE = 12'shx,

parameter logic signed [11:0] RESET_VALUE    = 12'shx

)(

input logic   CLK,

input logic   RST_N,

input logic   SE,

input logic   SI,

input logic   AER_IN_REQ,

input logic [7:0] AER_IN_ADDR,

input logic   AER_OUT_ACK,

output logic  SO,

output logic  AER_IN_ACK,

output logic  AER_OUT_REQ,

output logic [7:0] AER_OUT_ADDR

);

```

2. Simulation and Verification Results

Four DUT instances were instantiated with different parameter configs for positive weight behavior, negative weight behavior, maximum signed weight, and minimum signed weight. The testbench verifies both functional datapath behavior and AER handshake behavior.

DUT Configurations

DUT	Weight	Threshold	Reset	Primary purpose
dut_pos	+1	500	0	Normal positive update, threshold crossing, spike generation
dut_neg	-1	500	0	Signed negative arithmetic
dut_max	+127	2047	-123	Maximum signed weight/upper bound spike case
dut_min	-128	2047	0	Minimum signed weight/negative saturation

2.1 Reset and Idle Behavior

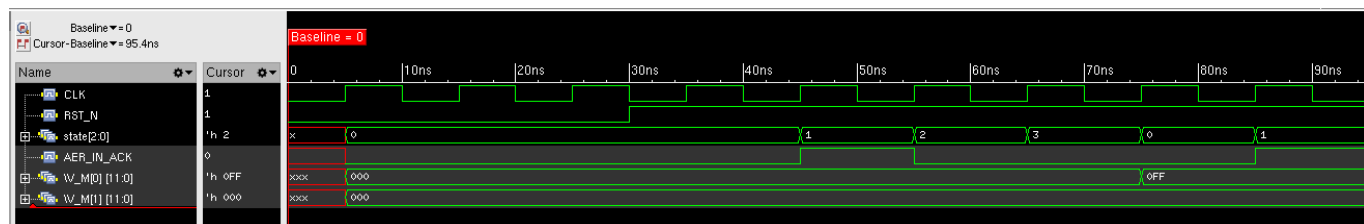
Purpose: Reset must place the block in a deterministic idle condition before any AER transaction begins.

Stimulus: Assert RST_N low, then release reset without applying an input event.

Expected behavior: The FSM is in IDLE, AER_IN_ACK and AER_OUT_REQ are low, representative neuron states are zero, and no unintended transaction begins.

Important Signals:

Signal	Expected behavior	Why it matters
CLK	Free running clock	Provides timing reference for reset release and FSM behavior.
RST_N	Low during reset, then rises	Shows when reset is applied and released.
state	IDLE after reset	Confirms controller returns to the idle state.
AER_IN_ACK	0	No input transaction should be acknowledged.
AER_OUT_REQ	0	No output spike should be requested.
V_M[0]	0	Representative neuron state is cleared.
V_M[1]	0	Second representative neuron state is also cleared.



2.2 Normal Positive Weight Neuron Update

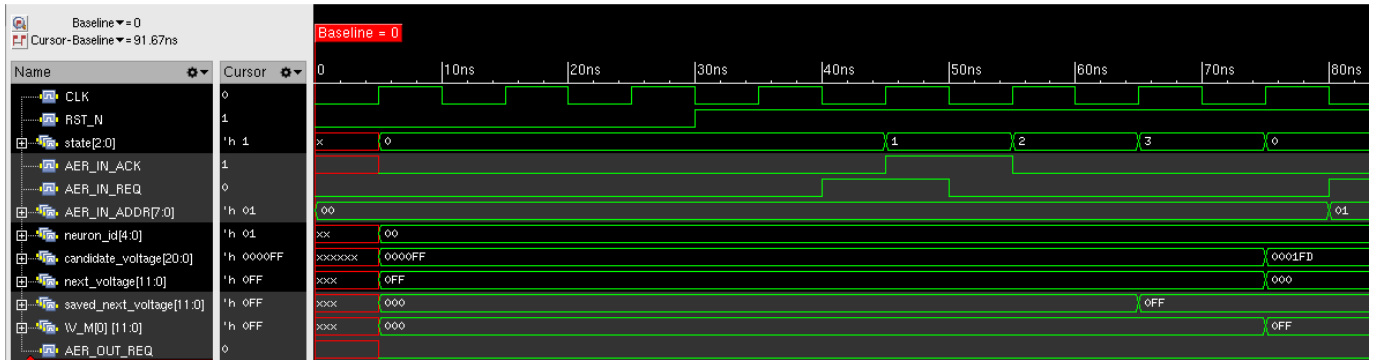
Purpose: A normal AER event must be accepted, processed through the shared datapath, written back to the selected neuron, and completed without generating a spike when the threshold is not reached.

Stimulus: Send a positive weight event to a neuron initially at 0 using dut_pos.

Expected behaviour: The input is acknowledged, the selected neuron is processed, candidate/next voltage evaluate to +255, V_M updates from 0 to 255, and AER_OUT_REQ remains low because $255 < 500$.

Important Signals:

Signal	Expected behavior	Why it matters
AER_IN_REQ	Pulses high for the input request	Marks the incoming AER transaction.
AER_IN_ADDR	Shows the selected neuron address	Confirms which neuron is targeted.
AER_IN_ACK	Pulses high after acceptance	Confirms the request was accepted.
state	0 -> 1 -> 2 -> 3 -> 0	Shows the normal non spiking FSM path.
neuron_id	Matches the requested neuron	Confirms correct temporary address to neuron selection.
candidate_voltage	+255	Shows the combinational datapath result.
next_voltage	+255	Shows the value chosen for write back.
saved_next_voltage	+255	Shows the transaction result captured by the FSM.
V_M[0]	0 -> 255	Confirms state write back.
AER_OUT_REQ	Remains 0	No spike should be generated below threshold.



2.3 All 32 Neurons Are Updateable

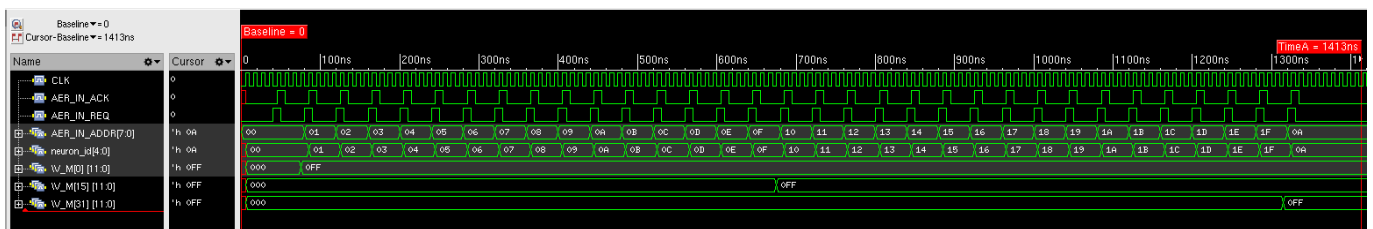
Purpose: Every entry in the 32 neuron state array must be individually selectable and writable under the current temporary neuron selection convention.

Stimulus: Sequentially apply requests with AER_IN_ADDR values 0x00 through 0x1F.

Expected behaviour: Each address is acknowledged and the corresponding V_M entry is updated. Representative first, middle, and last neurons should visibly change from 0 to 255.

Important Signals:

Signal	Expected behavior	Why it matters
AER_IN_REQ	Repeated request pulses	Shows a sequence of independent transactions.
AER_IN_ADDR	00 -> 01 -> ... -> 1F	Shows all 32 address values are exercised.
AER_IN_ACK	One ACK per accepted transaction	Confirms each request is accepted.
neuron_id	Tracks addresses 0-31	Confirms temporary selection logic reaches every neuron.
V_M[0]	0 -> 255	Representative first neuron.
V_M[15]	0 -> 255	Representative middle neuron.
V_M[31]	0 -> 255	Representative final neuron.



2.4 Independent State Between Different Neurons

Purpose: Updating one neuron must not corrupt the stored membrane voltage of another neuron.

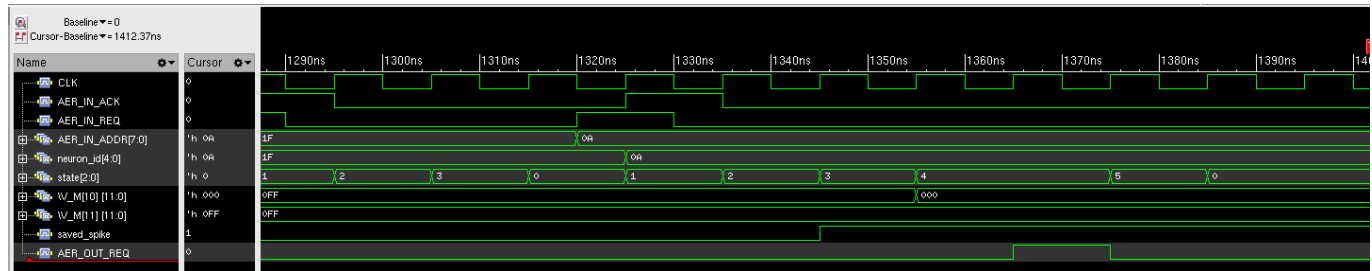
Stimulus: Start neurons 10 and 11 at 255, then issue another event only to neuron 10.

Expected behaviour: Neuron 10 crosses threshold and resets to 0 while neuron 11 remains at 255.

Important Signals:

Signal	Expected behavior	Why it matters
AER_IN_ADDR	0x0A	Targets neuron 10.
AER_IN_REQ	Pulses high	Starts the second update.
AER_IN_ACK	Pulses high	Confirms the event is accepted.

state	Processes neuron 10 through the FSM	Shows control sequencing.
neuron_id	10	Confirms the intended neuron is active.
V_M[10]	255 -> 0	Target neuron is updated/reset after spike.
V_M[11]	Remains 255	Non-target neuron is preserved.
saved_spike	Becomes 1	Confirms the neuron 10 transaction crossed threshold.
AER_OUT_REQ	Asserts	Spike generated for neuron 10.



2.5 Threshold Detection and Post Spike Reset

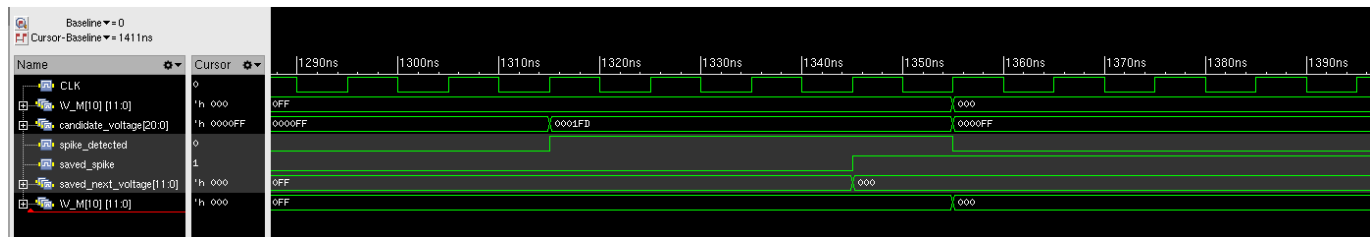
Purpose: The LIF datapath must detect when the updated membrane voltage reaches or exceeds threshold and replace the candidate value with RESET_VALUE.

Stimulus: Apply a second positive event to a neuron already at 255 using threshold 500 and reset 0.

Expected behaviour: The candidate is approximately 509, spike_detected asserts, saved_spike captures 1, saved_next_voltage becomes 0, and V_M is written with 0.

Important Signals:

Signal	Expected behavior	Why it matters
candidate_voltage	Approximately 509	Shows the pre threshold arithmetic result.
spike_detected	1 while candidate >= threshold	Live combinational threshold comparison.
saved_spike	1	Latched spike decision for the transaction.
saved_next_voltage	0	Latched reset value to be written.
V_M[10]	255 -> 0	Confirms post spike reset is committed.



2.6 Outgoing Spike and AER Address Generation

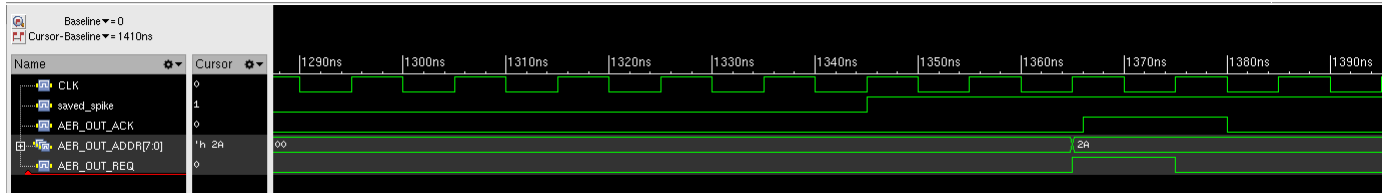
Purpose: A threshold crossing must produce an outgoing AER request with the expected Team-2 address encoding.

Stimulus: Use the neuron #10 threshold crossing from the previous test.

Expected behaviour: AER_OUT_REQ asserts and AER_OUT_ADDR becomes 0x2A: upper nibble 0x2 for Team 2 and lower nibble 0xA for the source neuron under the current interface convention.

Important Signals:

Signal	Expected behavior	Why it matters
saved_spike	1	Indicates that the transaction generated a spike.
AER_OUT_REQ	0 -> 1	Starts the outgoing AER transaction.
AER_OUT_ADDR	0x2A	Confirms expected tile/source encoding.
AER_OUT_ACK	Eventually asserts	Shows the downstream receiver acknowledges the event.



2.7 Negative Weight Arithmetic

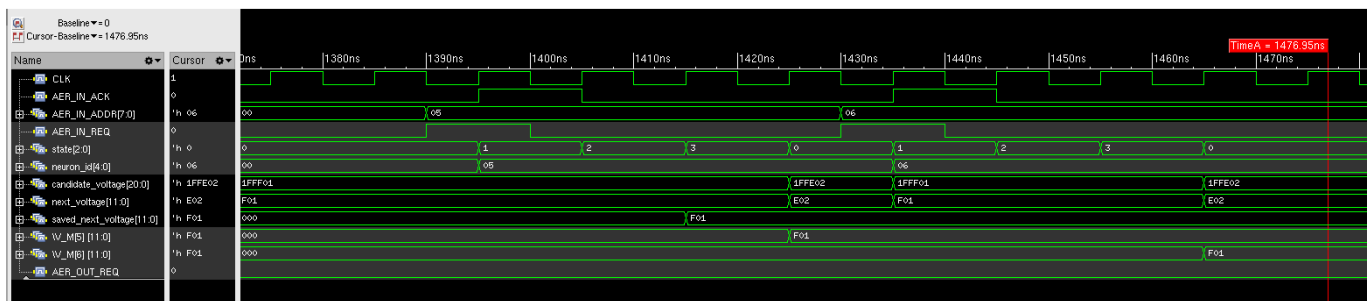
Purpose: The signed datapath must correctly handle a negative synaptic weight and update neuron state in the negative direction without falsely generating a spike.

Stimulus: Use dut_neg with $W = -1$ and send events to neurons 5 and 6.

Expected behaviour: Each selected neuron changes from 0 to -255 and AER_OUT_REQ remains low.

Important Signals:

Signal	Expected behavior	Why it matters
AER_IN_REQ	Request pulse for each neuron	Starts each negative weight transaction.
AER_IN_ADDR	0x05 then 0x06	Selects neurons 5 and 6.
AER_IN_ACK	ACK for each request	Confirms each transaction is accepted.
state	Normal non-spiking path	Shows controller sequencing.
neuron_id	5 then 6	Confirms target selection.
candidate_voltage	-255	Shows signed negative arithmetic.
next_voltage	-255	Shows selected write back value.
saved_next_voltage	-255	Latched transaction result.
V_M[5]	0 -> -255	First negative update.
V_M[6]	0 -> -255	Second independent negative update.
AER_OUT_REQ	Remains 0	Negative update must not cause a positive threshold spike.



2.8 Input Rejection While Datapath Is Busy

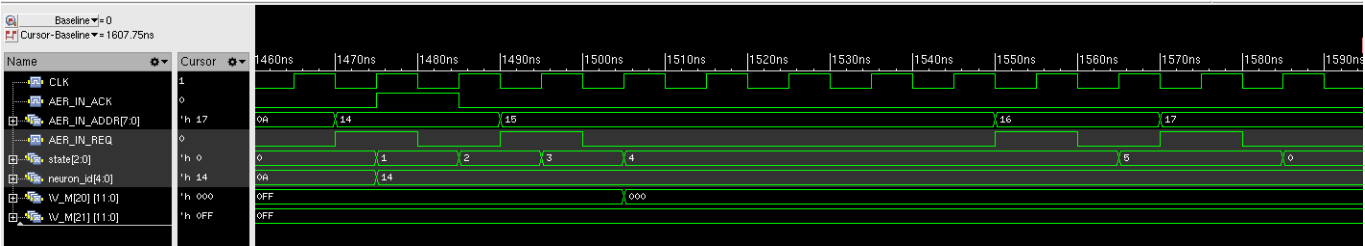
Purpose: Because the tile uses one shared arithmetic datapath, a second input event must not be accepted while another neuron update is still in progress.

Stimulus: Begin processing neuron 20, then assert another request targeting neuron 21 before the first transaction finishes.

Expected behaviour: The second request receives no acknowledgement and V_M[21] remains unchanged while neuron 20 completes.

Important Signals:

Signal	Expected behavior	Why it matters
AER_IN_REQ	Shows the attempted second request	Indicates new traffic arrives while busy.
AER_IN_ADDR	Changes to neuron 21	Identifies the rejected second target.
AER_IN_ACK	Remains low for the second request	Proves it is not accepted.
state	Non idle while second request is present	Shows the datapath/controller is busy.
neuron_id	Remains associated with active transaction	Shows the current event is preserved.
V_M[20]	Completes its active update	Confirms the original transaction finishes.
V_M[21]	Unchanged	Confirms the rejected request causes no state write.

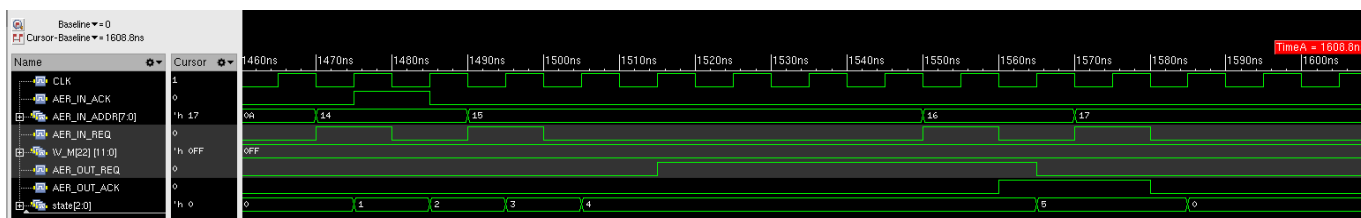


Stimulus: While AER_OUT_REQ is high and AER_OUT_ACK is low, apply a new input event for neuron 22.

Expected behaviour: AER_IN_ACK remains low and V_M[22] does not change until the output transaction completes.

Important Signals:

Signal	Expected behavior	Why it matters
AER_IN_REQ	Asserts during blocked output	Creates the attempted new input transaction.
AER_IN_ADDR	Targets neuron 22	Identifies the rejected target.
AER_IN_ACK	Remains 0	Confirms new input is not accepted.
V_M[22]	Unchanged	Confirms no hidden state update occurs.
AER_OUT_REQ	Remains 1	Shows the previous output event is still active.
AER_OUT_ACK	Remains 0 during rejection	Confirms output is still blocked.
state	Remains in output transaction state	Shows controller cannot return to input processing yet.



2.11 Correct Completion of the Output Handshake

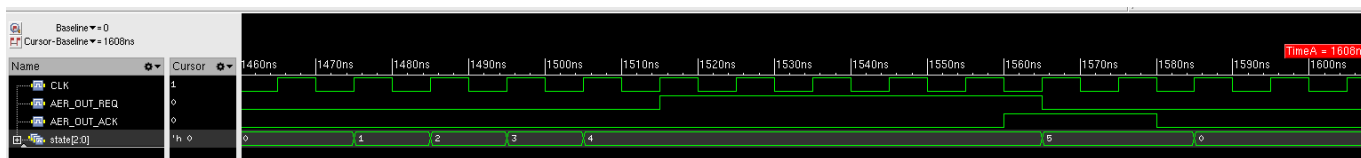
Purpose: The outgoing AER request/acknowledge sequence must complete in the expected four phase order and return the controller to IDLE.

Stimulus: Allow AER_OUT_ACK to assert after a pending AER_OUT_REQ, then release ACK.

Expected behaviour: The sequence is REQ up, ACK up, REQ down, ACK down, followed by state returning to IDLE.

Important Signals:

Signal	Expected behavior	Why it matters
AER_OUT_REQ	Rises first, then falls after ACK	Sender side of the handshake.
AER_OUT_ACK	Rises after REQ, then falls after REQ	Receiver side of the handshake.
state	SEND_SPIKE -> WAIT_OUTPUT_ACK_LOW -> IDLE	Confirms correct control sequencing and recovery.



2.12 Maximum Signed Weight

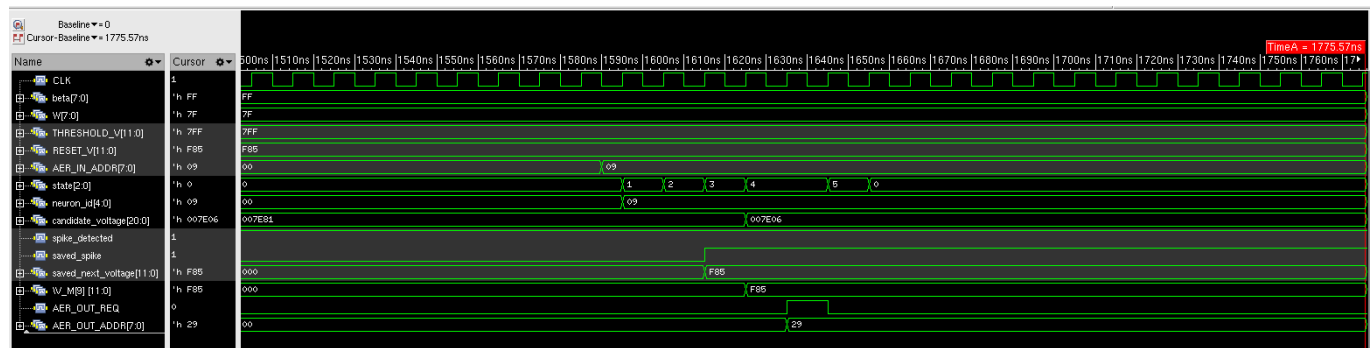
Purpose: The datapath must correctly handle the maximum legal signed 8 bit weight and produce the expected threshold crossing/spike behavior.

Stimulus: Use dut_max with W = +127, THRESHOLD = 2047, RESET = -123 and target neuron 9.

Expected behaviour: The candidate voltage exceeds threshold, the transaction records a spike, V_M[9] is written with -123, and an output event with address 0x29 is generated.

Important Signals:

Signal	Expected behavior	Why it matters
W	+127	Confirms maximum signed positive weight configuration.
THRESHOLD_V	2047	Shows upper configured threshold.
RESET_V	-123	Shows configured post spike reset value.
AER_IN_ADDR	0x09	Targets neuron 9.
state	Spiking FSM path	Shows control transitions through output generation.
neuron_id	9	Confirms target selection.
candidate_voltage	Large positive value > 2047	Shows upper bound arithmetic case.
spike_detected	1	Live threshold result.
saved_spike	1	Latched transaction spike decision.
saved_next_voltage	-123	Latched reset value.
V_M[9]	0 -> -123	Confirms post spike write back.
AER_OUT_REQ	Asserts	Outgoing event generated.
AER_OUT_ADDR	0x29	Expected Team2/source address.



2.13 Minimum Signed Weight and Negative Saturation

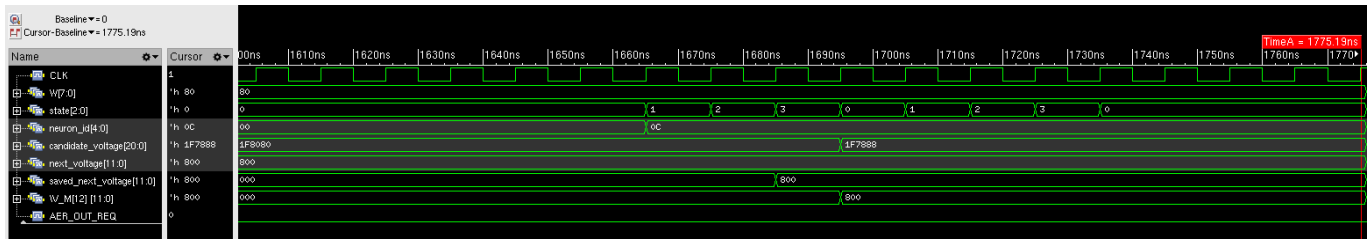
Purpose: The datapath must correctly handle the minimum legal signed 8 bit weight and clamp an out of range negative result instead of wrapping around.

Stimulus: Use dut_min with W = -128 and target neuron 12. Apply the minimum weight update twice.

Expected behaviour: The raw candidate falls below -2048, next_voltage and saved_next_voltage clamp to -2048, V_M[12] becomes -2048, and a second update leaves it at -2048 with no spike.

Important Signals:

Signal	Expected behavior	Why it matters
W	-128	Confirms minimum signed 8 bit weight configuration.
state	Normal non-spiking update path	Shows control flow.
neuron_id	12	Confirms target selection.
candidate_voltage	Below -2048	Shows out of range negative arithmetic result.
next_voltage	-2048	Shows lower saturation/clamp decision.
saved_next_voltage	-2048	Latched saturated write back value.
V_M[12]	0 -> -2048, then remains -2048	Proves saturation without wraparound.
AER_OUT_REQ	Remains 0	Negative saturation must not generate a positive threshold spike.



2.14 Verification Summary

Verification point	Expected result	Status
Reset / idle	Known state after reset	PASS
Normal positive update	0 -> 255, no spike	PASS
All 32 neurons	Every neuron entry updateable	PASS
Independent state	Non-target neuron unchanged	PASS
Threshold / reset	Threshold crossing causes reset	PASS
Outgoing spike	REQ/address generated correctly	PASS
Negative weight	0 -> -255, no spike	PASS
Busy input rejection	Second request not accepted	PASS
Output backpressure	REQ/address held stable	PASS
Input blocked by output	New request rejected	PASS
Handshake completion	REQ/ACK sequence returns to IDLE	PASS
Maximum weight	+127 threshold/spike/reset case	PASS
Minimum weight	-128 saturates at -2048	PASS

Testbench summary: (xrun.log uploaded to repo)

```

=====

TESTS RUN   : 344

TESTS FAILED : 0

RESULT      : PASS

=====

```

3. HDL Code

3.1 Implementation File (block2_digital_snn.sv)

```
`timescale 1ns/1ps // for simulation timing
```

```
module block2_digital_snn #(
```

```
    // Supply these values from the testbench for now.
```

```
    // Unknown defaults expose a missing test configuration in simulation.
```

```
    parameter logic [7:0]    BETA_VALUE    = 8'hxx,
```

```
    parameter logic signed [7:0] WEIGHT_VALUE = 8'shx,
```

```
    parameter logic signed [11:0] THRESHOLD_VALUE = 12'shx,
```

```

parameter logic signed [11:0] RESET_VALUE = 12'shx

)(
    input logic    CLK,
    input logic    RST_N,
    input logic    SE,
    input logic    SI,
    input logic    AER_IN_REQ,
    input logic [7:0] AER_IN_ADDR,
    input logic    AER_OUT_ACK,
    output logic    SO,
    output logic    AER_IN_ACK,
    output logic    AER_OUT_REQ,
    output logic [7:0] AER_OUT_ADDR
);

```

```

// Each membrane voltage is signed 12-bit Q4.8.

```

```

logic signed [11:0] V_M [0:31];

```

```

// The neuron currently being updated.

```

```

logic [4:0] neuron_id;

```

```

// Configuration used by the datapath.

```

```

logic [7:0] beta;

```

```

logic signed [7:0] W;

```

```

logic signed [11:0] THRESHOLD_V;

```

```

logic signed [11:0] RESET_V;

```

```

assign beta    = BETA_VALUE;

```

```

assign W       = WEIGHT_VALUE;

```

```

assign THRESHOLD_V = THRESHOLD_VALUE;

assign RESET_V    = RESET_VALUE;


// Datapath for V_candidate = beta * (V_M + W).

// Existing temporary interpretation: W represents whole voltage units.

logic signed [20:0] weight_q8;

logic signed [20:0] voltage_plus_weight;

logic signed [29:0] leaked_product;

logic signed [20:0] candidate_voltage;

logic spike_detected;

logic signed [11:0] next_voltage;


always_comb begin

    weight_q8 = $signed({13{W[7]}}, W) <<< 8;


    voltage_plus_weight =

        $signed(V_M[neuron_id]) + weight_q8;


    leaked_product =

        voltage_plus_weight * $signed({1'b0, beta});


    // Return from 16 fractional bits to eight.

    candidate_voltage = leaked_product >>> 8;


    spike_detected =

        candidate_voltage >= $signed(THRESHOLD_V);


    if (spike_detected) begin

        next_voltage = RESET_V;

```

```

end

else if (candidate_voltage > 21'sd2047) begin

    next_voltage = 12'sd2047;

end

else if (candidate_voltage < -21'sd2048) begin

    next_voltage = -12'sd2048;

end

else begin

    next_voltage = candidate_voltage[11:0];

end

end

```

```

typedef enum logic [2:0] {

    IDLE,

    WAIT_INPUT_REQ_LOW,

    READ_NEURON,

    WRITE_NEURON,

    SEND_SPIKE,

    WAIT_OUTPUT_ACK_LOW

} state_t;

```

```

state_t state;

```

```

logic [7:0] captured_addr;

logic signed [11:0] saved_next_voltage;

logic saved_spike;

```

```

integer i;

```

```
// Membrane-voltage scan chain:  
  
// SI -> V_M[0] -> ... -> V_M[31] -> SO.  
  
assign SO = V_M[31][11];
```

```
always_ff @(posedge CLK) begin
```

```
    if (!RST_N) begin
```

```
        state      <= IDLE;
```

```
        captured_addr  <= '0;
```

```
        neuron_id      <= '0;
```

```
        saved_next_voltage <= '0;
```

```
        saved_spike      <= 1'b0;
```

```
        AER_IN_ACK       <= 1'b0;
```

```
        AER_OUT_REQ       <= 1'b0;
```

```
        AER_OUT_ADDR      <= '0;
```

```
        for (i = 0; i < 32; i = i + 1)
```

```
            V_M[i] <= '0;
```

```
    end
```

```
    else if (SE) begin
```

```
        // Shift the membrane registers; pause normal FSM operation.
```

```
        V_M[0] <= {V_M[0][10:0], SI};
```

```
        for (i = 1; i < 32; i = i + 1)
```

```
            V_M[i] <= {V_M[i][10:0], V_M[i-1][11]};
```

```
    end
```

```
    else begin
```

```
case (state)
```

```
  IDLE: begin
```

```
    if (AER_IN_REQ) begin
```

```
      captured_addr <= AER_IN_ADDR;
```

```
      // TEMPORARY mapping retained from your code.
```

```
      // Replace when the input-address meaning is known.
```

```
      neuron_id <= AER_IN_ADDR[4:0];
```

```
      AER_IN_ACK <= 1'b1;
```

```
      state <= WAIT_INPUT_REQ_LOW;
```

```
    end
```

```
  end
```

```
  WAIT_INPUT_REQ_LOW: begin
```

```
    // Keep ACK asserted until the sender lowers REQ.
```

```
    if (!AER_IN_REQ) begin
```

```
      AER_IN_ACK <= 1'b0;
```

```
      state <= READ_NEURON;
```

```
    end
```

```
  end
```

```
  READ_NEURON: begin
```

```
    // Save the current combinational datapath result.
```

```
    saved_next_voltage <= next_voltage;
```

```
    saved_spike <= spike_detected;
```

```
    state <= WRITE_NEURON;
```

```
  end
```

WRITE_NEURON: begin

V_M[neuron_id] <= saved_next_voltage;

if (saved_spike)

state <= SEND_SPIKE;

else

state <= IDLE;

end

SEND_SPIKE: begin

if (!AER_OUT_REQ) begin

// Team 2 tile ID plus incoming event-source ID.

// This does not uniquely identify a Team 2 neuron.

AER_OUT_ADDR <= {4'b0010, captured_addr[3:0]};

AER_OUT_REQ <= 1'b1;

end

else if (AER_OUT_ACK) begin

AER_OUT_REQ <= 1'b0;

state <= WAIT_OUTPUT_ACK_LOW;

end

end

WAIT_OUTPUT_ACK_LOW: begin

if (!AER_OUT_ACK)

state <= IDLE;

end

default: begin

```

    AER_IN_ACK <= 1'b0;

    AER_OUT_REQ <= 1'b0;

    state    <= IDLE;

end

```

```

endcase

```

```

end

```

```

end

```

```

endmodule

```

3.2 Testbench File (block2_digital_snn_tb.sv)

```

`timescale 1ns/1ps

```

```

// this tb is AI generated, I will include the logs in this directory

```

```

module block2_digital_snn_tb;

```

```

    // =====

```

```

    // Verification-only parameter sets.

```

```

    // These values are chosen to exercise functionality; they are not

```

```

    // proposed final architectural constants.

```

```

    // =====

```

```

    localparam logic [7:0] BETA_TEST = 8'd255;

```

```

    // Normal positive-weight DUT: convenient threshold crossing after

```

```

    // two updates of the same neuron.

```

```

    localparam logic signed [7:0] POS_WEIGHT  = 8'sd1;

```

```

    localparam logic signed [11:0] POS_THRESHOLD = 12'sd500;

```

```

    localparam logic signed [11:0] POS_RESET   = 12'sd0;

```

```

// Normal negative-weight DUT.

localparam logic signed [7:0] NEG_WEIGHT  = -8'sd1;

localparam logic signed [11:0] NEG_THRESHOLD = 12'sd500;

localparam logic signed [11:0] NEG_RESET   = 12'sd0;


// Signed 8-bit maximum weight boundary.

// A very large positive candidate must cross the largest possible
// signed 12-bit threshold and therefore take the spike/reset path.

localparam logic signed [7:0] MAX_WEIGHT  = 8'sd127;

localparam logic signed [11:0] MAX_THRESHOLD = 12'sd2047;

localparam logic signed [11:0] MAX_RESET   = -12'sd123;


// Signed 8-bit minimum weight boundary.

// This produces a candidate below the signed 12-bit minimum and
// therefore exercises the negative saturation clamp to -2048.

localparam logic signed [7:0] MIN_WEIGHT  = -8'sd128;

localparam logic signed [11:0] MIN_THRESHOLD = 12'sd2047;

localparam logic signed [11:0] MIN_RESET   = 12'sd0;


// =====

// Common controls

// =====

logic CLK;

logic RST_N;

logic SE;

logic SI;


// Positive DUT interface

logic pos_in_req;

```

```
logic [7:0] pos_in_addr;  
  
logic    pos_out_ack;  
  
logic    pos_so;  
  
logic    pos_in_ack;  
  
logic    pos_out_req;  
  
logic [7:0] pos_out_addr;
```

```
// Negative DUT interface
```

```
logic    neg_in_req;  
  
logic [7:0] neg_in_addr;  
  
logic    neg_out_ack;  
  
logic    neg_so;  
  
logic    neg_in_ack;  
  
logic    neg_out_req;  
  
logic [7:0] neg_out_addr;
```

```
// Maximum-value DUT interface
```

```
logic    max_in_req;  
  
logic [7:0] max_in_addr;  
  
logic    max_out_ack;  
  
logic    max_so;  
  
logic    max_in_ack;  
  
logic    max_out_req;  
  
logic [7:0] max_out_addr;
```

```
// Minimum-value DUT interface
```

```
logic    min_in_req;  
  
logic [7:0] min_in_addr;  
  
logic    min_out_ack;
```

```

logic    min_so;

logic    min_in_ack;

logic    min_out_req;

logic [7:0] min_out_addr;


int tests_run  = 0;

int tests_failed = 0;


// =====

// DUT instances

// =====

block2_digital_snn #(
    .BETA_VALUE    (BETA_TEST),
    .WEIGHT_VALUE  (POS_WEIGHT),
    .THRESHOLD_VALUE (POS_THRESHOLD),
    .RESET_VALUE   (POS_RESET)
) dut_pos (
    .CLK      (CLK),
    .RST_N    (RST_N),
    .SE       (SE),
    .SI       (SI),
    .AER_IN_REQ (pos_in_req),
    .AER_IN_ADDR (pos_in_addr),
    .AER_OUT_ACK (pos_out_ack),
    .SO       (pos_so),
    .AER_IN_ACK (pos_in_ack),
    .AER_OUT_REQ (pos_out_req),
    .AER_OUT_ADDR (pos_out_addr)
);

```

```

block2_digital_snn #(
    .BETA_VALUE    (BETA_TEST),
    .WEIGHT_VALUE  (NEG_WEIGHT),
    .THRESHOLD_VALUE (NEG_THRESHOLD),
    .RESET_VALUE   (NEG_RESET)
) dut_neg (
    .CLK    (CLK),
    .RST_N  (RST_N),
    .SE     (SE),
    .SI     (SI),
    .AER_IN_REQ (neg_in_req),
    .AER_IN_ADDR (neg_in_addr),
    .AER_OUT_ACK (neg_out_ack),
    .SO       (neg_so),
    .AER_IN_ACK (neg_in_ack),
    .AER_OUT_REQ (neg_out_req),
    .AER_OUT_ADDR (neg_out_addr)
);

```

```

block2_digital_snn #(
    .BETA_VALUE    (BETA_TEST),
    .WEIGHT_VALUE  (MAX_WEIGHT),
    .THRESHOLD_VALUE (MAX_THRESHOLD),
    .RESET_VALUE   (MAX_RESET)
) dut_max (
    .CLK    (CLK),
    .RST_N  (RST_N),
    .SE     (SE),

```

```

.SI      (SI),

.AER_IN_REQ (max_in_req),

.AER_IN_ADDR (max_in_addr),

.AER_OUT_ACK (max_out_ack),

.SO      (max_so),

.AER_IN_ACK (max_in_ack),

.AER_OUT_REQ (max_out_req),

.AER_OUT_ADDR (max_out_addr)

);

```

```

block2_digital_snn #(

.BETA_VALUE  (BETA_TEST),

.WEIGHT_VALUE (MIN_WEIGHT),

.THRESHOLD_VALUE (MIN_THRESHOLD),

.RESET_VALUE  (MIN_RESET)

) dut_min (

.CLK      (CLK),

.RST_N    (RST_N),

.SE       (SE),

.SI       (SI),

.AER_IN_REQ (min_in_req),

.AER_IN_ADDR (min_in_addr),

.AER_OUT_ACK (min_out_ack),

.SO       (min_so),

.AER_IN_ACK (min_in_ack),

.AER_OUT_REQ (min_out_req),

.AER_OUT_ADDR (min_out_addr)

);

```

```
always #5 CLK = ~CLK;
```

```
// =====
```

```
// Checking / timing helpers
```

```
// =====
```

```
task automatic check(input bit condition, input string message);
```

```
    tests_run++;
```

```
    if (!condition) begin
```

```
        tests_failed++;
```

```
        $error("FAIL: %s", message);
```

```
    end
```

```
    else begin
```

```
        $display("PASS: %s", message);
```

```
    end
```

```
endtask
```

```
task automatic wait_clks(input int n);
```

```
    repeat (n) @(posedge CLK);
```

```
endtask
```

```
// Complete an input four-phase transaction. This task waits through
```

```
// WAIT_INPUT_REQ_LOW -> READ_NEURON -> WRITE_NEURON, so the state write
```

```
// has occurred when it returns. It is intended for non-spiking cases.
```

```
task automatic send_event_nonspiking(
```

```
    ref logic    req,
```

```
    ref logic [7:0] addr_bus,
```

```
    ref logic    in_ack,
```

```
    input logic [7:0] addr,
```

```
    input string  tag
```

```

);

begin

    @(negedge CLK);

    addr_bus = addr;

    req    = 1'b1;

    @(posedge CLK);

    #1;

    check(in_ack === 1'b1,

        $sformatf("%s: input address 0x%02h acknowledged", tag, addr));

    @(negedge CLK);

    req = 1'b0;

    // WAIT_INPUT_REQ_LOW, READ_NEURON, WRITE_NEURON

    wait_clks(3);

    #1;

    check(in_ack === 1'b0,

        $sformatf("%s: input ACK returned low", tag));

end

endtask

// Start an input transaction and wait through WRITE_NEURON. A spiking
// transaction will enter SEND_SPIKE; AER_OUT_REQ asserts one clock later.

task automatic send_event_to_write(

    ref logic    req,

    ref logic [7:0] addr_bus,

    ref logic    in_ack,

    input logic [7:0] addr,

```

```

    input string tag

);

begin

    @(negedge CLK);

    addr_bus = addr;

    req = 1'b1;

    @(posedge CLK);

    #1;

    check(in_ack === 1'b1,

        $sformatf("%s: input address 0x%02h acknowledged", tag, addr));

    @(negedge CLK);

    req = 1'b0;

    wait_clks(3);

    #1;

    check(in_ack === 1'b0,

        $sformatf("%s: input ACK returned low", tag));

end

endtask

// =====

// Main directed test sequence

// =====

initial begin : test_sequence

    int n;

    logic signed [11:0] neuron1_before;

```

CLK = 1'b0;

RST_N = 1'b0;

SE = 1'b0;

SI = 1'b0;

pos_in_req = 1'b0;

pos_in_addr = 8'h00;

pos_out_ack = 1'b0;

neg_in_req = 1'b0;

neg_in_addr = 8'h00;

neg_out_ack = 1'b0;

max_in_req = 1'b0;

max_in_addr = 8'h00;

max_out_ack = 1'b0;

min_in_req = 1'b0;

min_in_addr = 8'h00;

min_out_ack = 1'b0;

\$dumpfile("simulation_output.vcd"); // for simulation

\$dumpvars;

\$display("\n=====");

\$display(" block2_digital_snn multi-configuration testbench");

\$display(" POS: W=%0d TH=%0d RESET=%0d", \$signed(POS_WEIGHT), \$signed(POS_THRESHOLD),
\$signed(POS_RESET));

```

$display(" NEG: W=%0d TH=%0d RESET=%0d", $signed(NEG_WEIGHT), $signed(NEG_THRESHOLD),
$signed(NEG_RESET));

$display(" MAX: W=%0d TH=%0d RESET=%0d", $signed(MAX_WEIGHT), $signed(MAX_THRESHOLD),
$signed(MAX_RESET));

$display(" MIN: W=%0d TH=%0d RESET=%0d", $signed(MIN_WEIGHT), $signed(MIN_THRESHOLD),
$signed(MIN_RESET));

$display("=====\n");

// -----

// TEST 1: Reset / idle behavior for every configuration

// -----

$display("\n--- TEST 1: RESET / IDLE ---");

wait_clks(3);

@(negedge CLK);

RST_N = 1'b1;

@(posedge CLK);

#1;

check(pos_in_ack === 1'b0 && pos_out_req === 1'b0,
    "positive DUT reset leaves AER interface idle");

check(neg_in_ack === 1'b0 && neg_out_req === 1'b0,
    "negative DUT reset leaves AER interface idle");

check(max_in_ack === 1'b0 && max_out_req === 1'b0,
    "maximum DUT reset leaves AER interface idle");

check(min_in_ack === 1'b0 && min_out_req === 1'b0,
    "minimum DUT reset leaves AER interface idle");

for (n = 0; n < 32; n++) begin

    check(dut_pos.V_M[n] === 12'sd0,

        $sformatf("positive DUT reset clears neuron %0d", n));

```

```

    check(dut_neg.V_M[n] === 12'sd0,

        $sformatf("negative DUT reset clears neuron %0d", n));

    check(dut_max.V_M[n] === 12'sd0,

        $sformatf("maximum DUT reset clears neuron %0d", n));

    check(dut_min.V_M[n] === 12'sd0,

        $sformatf("minimum DUT reset clears neuron %0d", n));

end

// -----

// TEST 2: Positive weight + every neuron updateable

// beta=255, W=+1: from zero the current RTL produces +255.

// -----

$display("\n--- TEST 2: POSITIVE WEIGHT / ALL 32 NEURONS ---");

for (n = 0; n < 32; n++) begin

    send_event_nonspiking(pos_in_req, pos_in_addr, pos_in_ack,

        n[7:0], "POS");

    check(dut_pos.V_M[n] === 12'sd255,

        $sformatf("POS: neuron %0d updated from 0 to +255", n));

    check(pos_out_req === 1'b0,

        $sformatf("POS: neuron %0d did not spike below threshold", n));

end

// Verify independent retention after all 32 were targeted.

for (n = 0; n < 32; n++) begin

    check(dut_pos.V_M[n] === 12'sd255,

        $sformatf("POS: neuron %0d retained independent state", n));

end

```

```

// -----

// TEST 3: Two different neurons are independently addressable

// using the current temporary AER_IN_ADDR[4:0] mapping.

// -----

$display("\n--- TEST 3: TWO DIFFERENT NEURONS ---");

check(dut_pos.V_M[10] === 12'sd255 && dut_pos.V_M[11] === 12'sd255,

      "neurons 10 and 11 begin with the same known state");


// A second event on neuron 10 crosses the chosen threshold and resets

// neuron 10. Neuron 11 must remain unchanged.

send_event_to_write(pos_in_req, pos_in_addr, pos_in_ack, 8'd10, "POS-N10");

check(dut_pos.V_M[10] === POS_RESET,

      "second event updates only neuron 10 and applies post-spike reset");

check(dut_pos.V_M[11] === 12'sd255,

      "updating neuron 10 does not modify neuron 11");


@(posedge CLK); // SEND_SPIKE asserts request

#1;

check(pos_out_req === 1'b1,

      "second event on neuron 10 produces an outgoing spike");

check(pos_out_addr === 8'h2A,

      "spike address reflects Team-2 tile ID and source nibble A");


// Finish this spike before continuing.

pos_out_ack = 1'b1;

@(posedge CLK);

#1;

check(pos_out_req === 1'b0,

      "POS: output request drops after downstream ACK");

```

```

@(negedge CLK);

pos_out_ack = 1'b0;

@(posedge CLK);

#1;

// -----

// TEST 4: Negative weight datapath

// beta=255, W=-1: from zero the current RTL produces -255.

// -----

$display("\n--- TEST 4: NEGATIVE WEIGHT ---");

send_event_nonspiking(neg_in_req, neg_in_addr, neg_in_ack, 8'd5, "NEG-N5");

check(dut_neg.V_M[5] === -12'sd255,

      "negative weight decreases neuron 5 from 0 to -255");

check(neg_out_req === 1'b0,

      "negative update below threshold does not produce a spike");

send_event_nonspiking(neg_in_req, neg_in_addr, neg_in_ack, 8'd6, "NEG-N6");

check(dut_neg.V_M[6] === -12'sd255,

      "negative weight independently updates neuron 6");

check(dut_neg.V_M[5] === -12'sd255,

      "updating neuron 6 preserves neuron 5 state");

// -----

// TEST 5: Input backpressure while shared datapath is busy

// Use fresh POS neurons 20 and 21 (currently +255 after TEST 2).

// -----

$display("\n--- TEST 5: INPUT WHILE DATAPATH BUSY ---");

neuron1_before = dut_pos.V_M[21];

```

```

@(negedge CLK);

pos_in_addr = 8'd20;

pos_in_req = 1'b1;

@(posedge CLK);

#1;

check(pos_in_ack === 1'b1,

    "busy test first request is accepted");


@(negedge CLK);

pos_in_req = 1'b0;

@(posedge CLK); // WAIT_INPUT_REQ_LOW -> READ_NEURON

#1;

check(pos_in_ack === 1'b0,

    "busy test first handshake completes");


// Present a second request during READ_NEURON.

@(negedge CLK);

pos_in_addr = 8'd21;

pos_in_req = 1'b1;

@(posedge CLK); // READ_NEURON executes

#1;

check(pos_in_ack === 1'b0,

    "second input is not acknowledged while datapath is busy");

check(dut_pos.V_M[21] === neuron1_before,

    "busy-time request does not modify second neuron");


@(negedge CLK);

pos_in_req = 1'b0;

@(posedge CLK); // WRITE_NEURON for neuron 20

```

```

#1;

check(dut_pos.V_M[20] === POS_RESET,

    "first busy-test neuron completes its threshold/reset update");

check(dut_pos.V_M[21] === neuron1_before,

    "unaccepted busy-time event leaves neuron 21 unchanged");


// -----

// TEST 6: Threshold spike + blocked output + blocked new input

// -----

$display("\n--- TEST 6: OUTPUT BACKPRESSURE ---");

@(posedge CLK); // SEND_SPIKE raises AER_OUT_REQ

#1;

check(pos_out_req === 1'b1,

    "threshold crossing generates AER_OUT_REQ");

check(pos_out_addr === 8'h24,

    "output spike address is stable for source nibble 4");


repeat (3) begin

    @(posedge CLK);

    #1;

    check(pos_out_req === 1'b1,

        "AER_OUT_REQ remains high while downstream ACK is low");

    check(pos_out_addr === 8'h24,

        "AER_OUT_ADDR remains stable while output is blocked");

end


// Attempt another input while output request is blocked.

@(negedge CLK);

pos_in_addr = 8'd22;

```

```

pos_in_req = 1'b1;

@(posedge CLK);

#1;

check(pos_in_ack === 1'b0,

    "new input is rejected while outgoing spike is blocked");

check(dut_pos.V_M[22] === 12'sd255,

    "blocked input does not alter its target neuron");

@(negedge CLK);

pos_in_req = 1'b0;


// Complete output handshake.

pos_out_ack = 1'b1;

@(posedge CLK);

#1;

check(pos_out_req === 1'b0,

    "AER_OUT_REQ drops after downstream ACK");


// Still blocked until ACK returns low.

@(negedge CLK);

pos_in_addr = 8'd23;

pos_in_req = 1'b1;

@(posedge CLK);

#1;

check(pos_in_ack === 1'b0,

    "input remains blocked until AER_OUT_ACK returns low");


@(negedge CLK);

pos_in_req = 1'b0;

pos_out_ack = 1'b0;

```

```

@(posedge CLK);

#1;

// -----

// TEST 7: Maximum signed weight (+127)

// With beta=255, the candidate from zero is far above +2047.

// Because spike detection occurs BEFORE saturation in the RTL and

// +2047 is the largest representable threshold, positive overflow

// necessarily follows the spike/reset path.

// -----

$display("\n--- TEST 7: MAXIMUM SIGNED WEIGHT (+127) ---");

send_event_to_write(max_in_req, max_in_addr, max_in_ack, 8'd9, "MAX");


check(dut_max.saved_spike === 1'b1,

    "maximum positive weight is detected as threshold crossing");

check(dut_max.V_M[9] === MAX_RESET,

    $sformatf("maximum positive case writes configured reset value %0d",
        $signed(MAX_RESET)));


@(posedge CLK); // SEND_SPIKE

#1;

check(max_out_req === 1'b1,

    "maximum positive-weight case generates a spike");

check(max_out_addr === 8'h29,

    "maximum positive-weight spike carries expected address");


max_out_ack = 1'b1;

@(posedge CLK);

#1;

```

```

check(max_out_req === 1'b0,

    "maximum-case spike request completes on ACK");

@(negedge CLK);

max_out_ack = 1'b0;

@(posedge CLK);

#1;


// -----

// TEST 8: Minimum signed weight (-128) / negative saturation

// From zero, beta=255 and W=-128 create a candidate below -2048,

// so the datapath must clamp the stored membrane voltage to -2048.

// -----

$display("\n--- TEST 8: MINIMUM SIGNED WEIGHT (-128) ---");

send_event_nonspiking(min_in_req, min_in_addr, min_in_ack, 8'd12, "MIN");


check(dut_min.V_M[12] === -12'sd2048,

    "minimum negative weight saturates membrane voltage at -2048");

check(min_out_req === 1'b0,

    "negative saturation does not spuriously generate a spike");


// Apply the minimum weight again. The state must remain clamped,

// proving that the lower boundary does not wrap around.

send_event_nonspiking(min_in_req, min_in_addr, min_in_ack, 8'd12, "MIN-REPEAT");

check(dut_min.V_M[12] === -12'sd2048,

    "repeated minimum update remains clamped at -2048 without wraparound");

check(min_out_req === 1'b0,

    "repeated negative saturation still does not generate a spike");


// -----

```

```

// TEST 9: Recovery after completed output handshake

// Use NEG DUT because it is currently idle and produces no spike.

// -----

$display("\n--- TEST 9: RECOVERY / CONTINUED OPERATION ---");

send_event_nonspiking(neg_in_req, neg_in_addr, neg_in_ack, 8'd7, "NEG-RECOVERY");

check(dut_neg.V_M[7] === -12'sd255,

      "DUT continues accepting new transactions after earlier tests");


// -----

// Summary

// -----

$display("\n=====");

$display(" TESTS RUN   : %0d", tests_run);

$display(" TESTS FAILED : %0d", tests_failed);

if (tests_failed == 0)

    $display(" RESULT   : PASS");

else

    $display(" RESULT   : FAIL");

$display("=====\\n");


if (tests_failed != 0)

    $fatal(1, "block2_digital_snn testbench failed");


$finish;

end


// Simulation watchdog.

initial begin

    #200000;

```

```
$fatal(1, "Simulation timeout");
```

```
end
```

```
endmodule
```

4. AI Log File

Session Report: SystemVerilog SNN Tile Testbench Development

1. Task or Question Worked On

[My decision / project work]

I worked on creating and refining a self-checking SystemVerilog testbench for the Team 2 digital SNN tile, block2_digital_snn. The goal was to verify the functionality of the block before the final neuron-selection scheme and final threshold/reset/weight values are fixed.

The main behaviors I wanted to test were:

- whether all 32 neurons can be updated,
- whether positive and negative weights are handled correctly,
- whether threshold crossings generate spikes,
- whether post-spike reset behavior works,
- whether the design blocks new input while busy,
- whether outgoing AER backpressure is handled correctly,
- whether minimum and maximum signed weight cases behave correctly.

2. Main Steps in My Reasoning

[My reasoning / decisions]

I decided that the testbench should focus on functional verification rather than final parameter tuning. Because neuron selection is still unresolved, I treated the current neuron-index mapping only as a temporary mechanism to prove that each of the 32 neuron-state entries is accessible.

I also decided that a single DUT instance was not enough because weight is a module parameter. To test positive and negative behavior in the same simulation, I needed multiple DUT instances with different parameter values.

I identified four useful test configurations:

- a normal positive-weight DUT,
- a normal negative-weight DUT,
- a maximum signed positive-weight DUT,
- a minimum signed negative-weight DUT.

I also wanted the testbench to be self-checking, so the testbench compares expected and actual behavior instead of relying only on manual waveform inspection.

[AI-generated assistance]

The AI helped translate these goals into directed test cases and suggested using multiple parameterized DUT instances so that positive, negative, minimum, and maximum weight cases could all be tested in one simulation.

3. Prompts or Approaches Tried

[My prompts / approaches]

I asked the AI to:

- implement a testbench for the supplied SystemVerilog design,
- test the full event-processing flow of the SNN tile,
- ignore the unresolved final neuron-selection policy and simply prove that every neuron is updateable,
- avoid treating the current threshold, reset voltage, and weight values as final design choices,
- rewrite the testbench to include separate DUT instances for positive and negative weights,
- add minimum and maximum signed-value tests.

[AI-generated assistance]

The AI produced a self-checking testbench structure and later rewrote it to use multiple DUT instances with different parameter values.

4. Important AI Responses or Suggestions

[AI-generated]

The AI suggested the following four DUT configurations:

- dut_pos with $W = +1$,
- dut_neg with $W = -1$,
- dut_max with $W = +127$,
- dut_min with $W = -128$.

The AI suggested using arbitrary but convenient threshold/reset values strictly for verification so that the block could be driven through both non-spiking and spiking cases.

The AI also suggested using hierarchical checks such as `dut.V_M[n]` because the current block does not expose neuron membrane voltage through a functional output port. This allows the testbench to directly verify that each neuron state was actually updated.

The AI also identified an RTL-specific limitation in the maximum-positive test: because threshold detection occurs before the positive saturation branch, the +127 case is best interpreted as a maximum signed-weight threshold/spike/reset test rather than a direct positive-saturation test.

5. What I Accepted, Changed, or Rejected

[My decisions]

I accepted the multi-DUT architecture because it allows all required weight-sign and boundary cases to be exercised in one testbench.

I accepted the use of internal neuron-state signals for verification because otherwise it would be difficult to prove that all 32 neurons are being updated correctly.

I accepted arbitrary verification-only parameter values because the current goal is to validate functionality rather than finalize the biological model.

I kept the four-DUT setup and did not add additional DUT instances because the positive, negative, maximum, and minimum cases already cover the required functional categories.

I also kept the testbench self-checking so that it can automatically report whether each directed test passes or fails.

6. Remaining Questions or Limitations

[My assessment / unresolved items]

The final neuron-selection and AER address mapping is still unresolved. The current testbench only proves that all 32 neuron-state entries are accessible under the temporary mapping.

The threshold, reset voltage, beta value, and weight values used by the testbench are verification values and are not intended to define the final design.

The testbench uses white-box checks on internal signals such as V_M, so it is not purely interface-based verification.

The maximum positive-weight test verifies correct threshold crossing, spike generation, and reset behavior, but it does not independently exercise positive saturation because of the current ordering of the RTL logic.